

acknowledgement (ACK) mechanism enables synchronisation, error detection, and automatic packet retransmission.

At the transmitter, the Python script (see Fig. 3) opens the selected file in binary mode, determines its size, and establishes communication through a short handshake. It then sends a <START> marker followed by metadata containing the file size and a fixed packet size of 128 bytes. The file is divided into 128-byte packets, each consisting of a header (0xAA, 0x55), a packet number, a data length, a payload, and an 8-bit checksum (sum of the data bytes modulo 256). The packets are transmitted through the TX pin, which drives a BC547 transistor to switch the laser diode at high speed for optical data transmission. Fig. 3

shows a snippet of the transmitter Python code.

After each packet, the transmitter waits for receiver feedback. A valid acknowledgement (V) triggers transmission of the next packet, while a negative acknowledgement (N) or a timeout causes the packet to be retransmitted. This ensures reliable data transfer despite noise, optical misalignment, or temporary link interruptions. After all packets have been received successfully, the transmitter sends an <END> marker to complete the file transfer.

On the receiver side (Fig. 4), a reverse-biased BPW34 photodiode detects the incoming optical signal and converts it into a voltage proportional to the light intensity. An LM393 comparator converts this analogue signal into a clean

digital UART signal. The Python receiver continuously monitors the serial port, waits for the <START> marker, reads the file metadata, and opens the output file in binary write mode. Fig. 4 shows a snippet of the receiver Python code.

Each received packet is verified by checking the header (0xAA, 0x55), packet number, payload length, and checksum. The checksum is recalculated and compared with the received value to ensure data integrity. If the packet is valid and in sequence, its payload is written to the file and a valid acknowledgement (V) is sent to the transmitter. If the checksum fails, a negative acknowledgement (N) is returned, requesting retransmission. Duplicate packets are ignored but still acknowledged to maintain synchronisation.

EFY note. Before running the Python script, verify the correct COM port in device manager under ports (COM and LPT), and ensure that the source and destination file paths are correctly configured.

Construction and testing

The actual-size PCB of the Li-Fi transceiver module (see Fig. 2) is shown in Fig. 5, and its component layout is shown in Fig. 6. After assembling the circuit on the PCB, mount it inside a suitable enclosure with an opening for the laser beam so that the light falls directly on the photodiode. Connect the PCB to a laptop or desktop computer through the USB interface.

Alternatively, the bi-directional Li-Fi system may also be assembled using standard breadboards, as shown in the author's prototype (Fig. 1).

To test the system, assemble two identical Li-Fi transceiver PCBs. Place the two modules at separate locations with a clear line of sight so that the laser beam from each module reaches the photodiode of the other. This arrangement simulates a practical bi-directional communication link. Ensure that the laser transmitter of each module is aligned with the receiver of the opposite module.

```
import serial
import time
import os

PORT = "COM31"
BAUD = 520000
PACKET_SIZE = 128

ser = serial.Serial(PORT, BAUD, timeout=0.5)
time.sleep(0.5)
ser.reset_input_buffer()
ser.reset_output_buffer()

path = r"C:\Users\Abhay Verma\Desktop\testfile.zip"
with open(path, "rb") as f:
    data = f.read()

filesize = len(data)
print("File size:", filesize)

def checksum(packet):
    return sum(packet) % 256

ser.write(b"abc\n")
time.sleep(0.001)
ser.write(b"abc\n")
ser.write(b"<START>\n")
ser.write(str(filesize).encode() + b"\n")
ser.write(str(PACKET_SIZE).encode() + b"\n")

packet_no = 0
i = 0

while i < filesize:
    chunk = data[i : i + PACKET_SIZE] # Prepare chunk
    header = bytes([0xAA, 0x55])
    length = len(chunk)

    packet = header + bytes([packet_no]) + bytes([length]) + chunk
    crc = checksum(packet)
    packet += bytes([crc])

    ser.write(packet) # Send packet
    ser.flush()

    feedback = ser.read(1) # Wait for feedback (blocks up to 0.5s due to timeout)

    if feedback == b'V': # Verify
        packet_no = (packet_no + 1) % 256
        i = i + len(chunk)
```

Fig. 3: Snippet of the transmitter Python code